Team #88 : Final Report

Problem Statement 1: AI Incident Report Assistant (Qualcomm QIDK) – *Speak. Understand. Extract. Review. Submit. Fully offline.*

1. Solution

We built a **voice-driven, self-improving incident reporting assistant that runs entirely on a Qualcomm QIDK** (Snapdragon 8 Elite `SM8750P`, Hexagon V79 NPU, Android 15). The user speaks naturally, in English, Hindi, Telugu or code-mixed speech. The app transcribes it, extracts the report with one on-device multimodal model (Gemma 4 E2B), asks only for what is still missing, fills a review form, reads the result back and saves it. Every model runs on the device **without any cloud APIs**, and the app works in airplane mode.

Our principle: the model proposes, the code verifies. The LLM only turns language into fields. Grounding, severity, which question to ask, when to stop and what to say about emergencies are deterministic, unit-tested Kotlin. This removed hallucinated fields and made the dialogue predictable. The app is native Kotlin / Jetpack Compose (MVVM) with JNI wrappers for llama.cpp and sherpa-onnx.

2. Approach and architecture

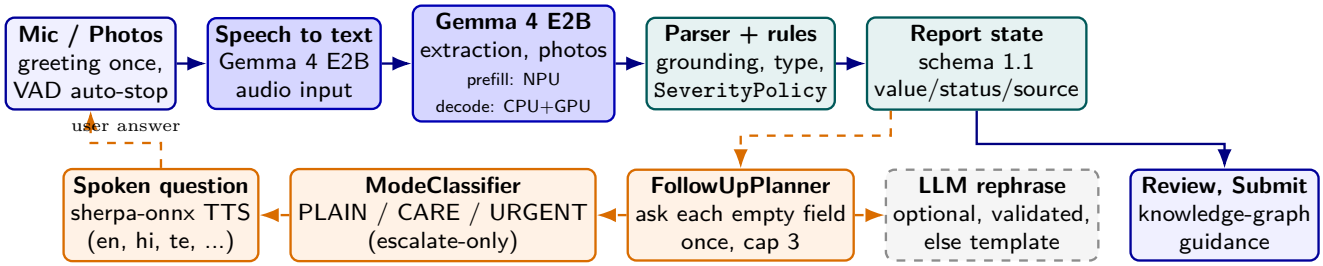


Figure 1: Architecture. Blue: Gemma 4 E2B (prefill on the NPU, decode on CPU+GPU); green: deterministic Kotlin; orange: follow-up dialogue (dashed: spoken loop); grey: optional.

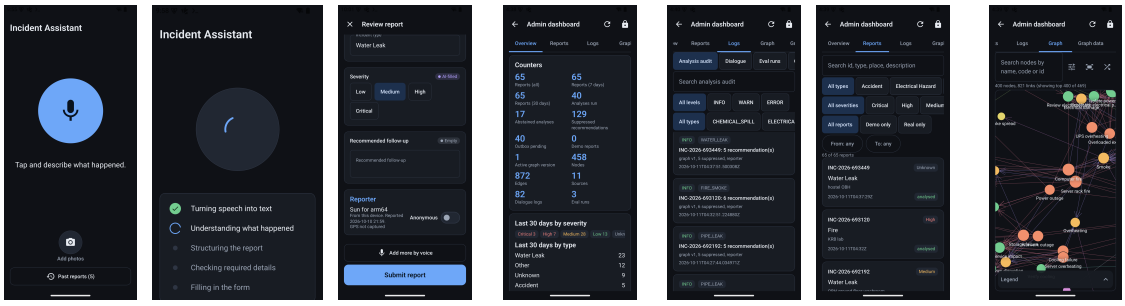
Stage	Model / component (all on-device)	Runtime
Speech to text	Gemma 4 E2B native audio input (hi, te, ta, as, or, en), no separate ASR model	llama.cpp b11552 JNI
Extraction	**Gemma 4 E2B** (q4_0): speech, photos and text in one model. **Prefill on the Hexagon NPU** (1,450 tok/s vs 320 on CPU); **decode on CPU+GPU** (35 tok/s vs 19 on NPU)	NPU prefill, CPU+GPU decode
Rules and dialogue	`SeverityPolicy`, grounding, mode classifier, planner, validator	Kotlin (pure, JVM-tested)
Speech output	Piper (English) + AI4Bharat Rasa (Indic scripts), Android TTS fallback	sherpa-onnx
Post-incident guidance	Offline knowledge graph: 458 nodes, 872 edges, 11 sources	Kotlin
Memory and admin	Persistent memory of all reports, past-data dashboard, event-graph visualization, self-improving agent	Kotlin, on-device storage

Key design decisions. (1) **One model, Gemma 4 E2B**, for speech, photos and text, with each phase on the fastest processor we measured (below). (2) **Small typed tools**, deterministic decoding (`temp 0.1, top-k 1`), few-shot turns and answer pre-filling for valid JSON. (3) **Free-text description built in code** from verified fields, because the model invented details in it. (4) **Ask-once dialogue**: no retry or "could you repeat" prompts; unanswered fields are flagged on the review screen. (5) **Severity from a rulebook** (5 tiers from 8 safety, healthcare, traffic and IT studies) with hard floors that only raise severity. (6) **JNI instead of a local server** for llama.cpp, so nothing in the app depends on a network socket.

3. Features

Feature	What it does
Voice or voice + photos	One-frame mic button (timer, waveform, processing orb). Gallery button attaches up to 3 photos; each photo is described and tagged by Gemma, and the description enters the extraction prompt.

Image-only mode (accessibility for mute and speech-impaired users)	Speaking is optional: “Use photos only” runs a silent session with no greeting, no questions and no microphone, so a user who cannot speak can still file a complete report. Gemma extracts **every detail visible in the image** (scene, objects, damage, hazards, people, visible text, location cues); a separate photo-only prompt derives type and severity from visible danger, and generic locations (“indoor”) are dropped. The report opens in Review with low-confidence fields highlighted; severity floors and the emergency flag still apply.
Natural speech	Fillers, self-corrections, any field order, Hinglish and romanised Telugu. Unmentioned fields stay empty and are never invented.
Multilingual	Hindi, Telugu, Tamil, Assamese, Odia audio in; report fields in English. Hindi and Telugu get their own questions, read-back, closing and form read-out.
Grounded extraction	Each value must trace to the user’s words. Slight ASR errors snap to the user’s wording (“catering stairs” → “cafeteria stairs”). Type grounding re-labels unsupported types.
Safety net	Keyword rules (English, Hindi, Telugu) so fire, gas and injury are never dropped as “no incident”; negation guard (“no one was hurt” is not an injury).
Follow-up dialogue	Spoken greeting once, then **only the empty fields, each asked at most once**, hands-free (mic restarts after each question). Tone modes `PLAIN` / `CARE` / `URGENT` (escalate-only). Hard cap of 3 questions.
Rephrasing with a guard	Optional LLM rewrite of a question, accepted only if it has one `?`, $\leq$20 words, no blame or “why”, within 1.5 s; otherwise the template is used.
Severity and emergency	Final severity = max(model hint, rule floors). Fire, gas and electrical $\geq$ High; `URGENT` plus an escalation cue or injury = Critical. Red emergency banner, one spoken safety check, “call for help” card with contact numbers that open the dialer.
Honest read-back	Summary of type, location and time; says “help is on the way” only if an alert was sent.
Review form	12 fields in 4 groups; badges for AI-filled, your answer, edited and empty; unsure or unclear fields highlighted; edit anything; “add more by voice”.
Offline voice	Spoken questions and confirmations through sherpa-onnx; one sentence may use two voices (English place name inside a Hindi sentence).
Knowledge-graph guidance	After submit: approved, ranked next steps (immediate, operational, recovery, prevention) from an offline graph; the LLM may only rephrase, and Kotlin removes unsafe, medical or expired items. History detection spots repeat incidents.
Admin dashboard and persistent memory	PIN-gated in-app dashboard; every report is kept in persistent on-device memory. The admin can open (i) **logs** of every report and its analysis, searchable and filterable; (ii) **report statistics** by day, type and severity; (iii) **graph-based incident visualisation**: an interactive event graph of the **direct and indirect events caused by an incident** (leak → short circuit → outage), highlighting the path behind each recommendation. A knowledge editor supports approve, import, export and rollback.
Self-improving agent	An agent that learns from past reports and admin corrections and **adapts to the user** over time.
Learned dictionary	A **dictionary learned from the user** fixes mispronunciations and speech-recognition errors: a corrected word or place name is remembered and applied to later transcripts, so the same mistake is not repeated.
Evaluation built in	Dev menu runs the eval set (WER, field match, mapping, dialogue, latency) and writes a results file; Mock AI keeps the UI demoable.

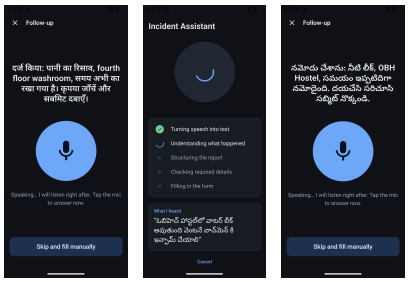


(a) Home (b) Processing (c) Review (d) Admin home (e) Logs (f) Statistics (g) Incident graph

Figure 2: The app on the QIDK (real model, no network): reporting flow (a–c) and admin dashboard (d–g).

4. Multilingual support

Users can speak Hindi, Telugu, Tamil, Assamese, Odia or English, including code-mixed speech; Gemma transcribes the audio on the device and the report fields are written in English. Questions, read-backs and the closing line are shown and spoken in the user's own language (Hindi and Telugu), and a single sentence may mix scripts and voices, such as an English place name ("fourth floor washroom", "OBH Hostel") inside a Hindi or Telugu sentence. A Telugu report of a water leak at a hostel is understood and filled in the same four-second pipeline as an English one.



(a) Hindi (b) Telugu (c) Telugu

Figure 3: Hindi and Telugu: spoken read-back with a mixed-script place name (a, c) and the Telugu transcript heard by the model (b).

5. Results

R1. Rule layer (35-case development set, 7 categories: clean, messy, missing information, non-incident, Hinglish, ASR noise, negation). The deterministic rules lifted field accuracy from 93.6% to **100%** and brought invented fields from 1 to **0** (measured with an earlier extraction model; the rules do not depend on the model). Because they were written after seeing this set, we treat 100% as optimistic and score a held-out set separately. All 6 Hinglish cases pass. Romanised Telugu was weak at first (fire, gas, injury returned as "no incident"); a Telugu/Hindi keyword safety net fixed every failed case.

R2. Speed on the QIDK (Gemma 4 E2B, model loaded once): we assigned each phase to the processor that measured fastest.

Phase	NPU	CPU / CPU+GPU	Our choice
Prefill (prompt processing)	**1,450 tok/s**	320 tok/s (CPU)	**NPU**, 4.5× faster
Decode (token generation)	19 tok/s	**35 tok/s** (CPU+GPU)	**CPU+GPU**, 1.8× faster
End of speech → filled form	**about 4 s** end to end for a spoken report		

R3. Spoken output in Hindi and Telugu (synthesise, transcribe back with Gemma, count matching words): location question 1.00 (hi) / 0.79 (te); safety check 1.00 / 0.90; review read-out 0.89 / 0.95; with an English place name inside 0.71 / 0.75.

R4. Image-only reports (QIDK, 6 scene photos, no speech): **6/6 runs ok, field accuracy 11/12, every severity floor met, 0 questions asked**. Five of six types matched; foam from a toilet was read as a spill, not a water leak. About 10 s per photo with the CPU encoder (7.1–7.7 s describe, 2.5–2.8 s extract). The GPU encoder is faster (2.4–4.5 s) but gave unstable descriptions; a one-call image-only path misread the burnt lab, so neither is used. Knowledge-graph analysis of a photo-only report still returns 2–3 immediate actions.

R5. Testing.

- **886 unit tests** in the suite (375 verified passing at the M2 snapshot): mode classifier and negation guard, follow-up planner (every priority rule, ask-once, caps), phrase validator, parser, output mapper, severity policy, flow reducer, evaluation metrics, knowledge-graph engine and voice routing.
- **14 scripted dialogue scenarios** pass with fake speech I/O, and again with 6 fake rephrasers (good text, "why", two questions, "ask again", null, timeout); all bad outputs fall back to the template. After every scenario: no field asked twice, no line spoken twice, mode never decreases.
- **24/24 rule self-checks** on real device outputs; on-device tests for the Gemma backend, sherpa TTS and the audio eval run.
- **Benchmarks:** 35-case typed set; 30-case end-to-end eval set; **office severity benchmark of 30 cases in 6 languages** (English, Hindi, Assamese, Odia, Telugu, Tamil; 180 narrations with Roman transliterations) with gold type, severity 0–4, urgency U0–U3, status and known traps; a 100-narrative labelled severity dataset with a rulebook.